MINIBANK
JAVA PRACTICAL REPORT

Student ID	D26CE163
Subject Code	CEUE203
Subject Name	Object Oriented Programming
Project Name	MiniBank

Academic Year: ___2026-2027___

Project Introduction

MiniBank is a simple console-based banking application developed in Java. It manages bank accounts, supports deposits and withdrawals, displays interest information, stores account data, records transactions, and demonstrates important Java concepts.

Main Features

- Create and display different types of bank accounts.
- Deposit and withdraw money with exception handling.
- Save account data and write transaction logs.
- Search, sort, and store accounts using Java collections.
- Demonstrate multithreading, thread pools, and deadlock prevention.
- Use a generic repository to save, search, list, count, and delete entities.

INDEX

Sr. No.	Practical / Content	Page No.
1	**Project Introduction and Main Features**	**4**
2	**Practical 1: Java Basics**	**4**
3	**Practical 2: Classes and Objects**	**5**
4	**Practical 3: Constructors and Encapsulation**	**6**
5	**Practical 4: Inheritance**	**7**
6	**Practical 5: Abstraction and Polymorphism**	**8**
7	**Practical 6: Interfaces and Lambda Expressions**	**9**
8	**Practical 7: Annotations and Reflection**	**10**
9	**Practical 8: Exception Handling**	**11**
10	**Practical 9: Multithreading**	**12**
11	**Practical 10: Thread Pools, Producer–Consumer, and Deadlock**	**13**
12	**Practical 11: File Persistence and Reports**	**15**
13	**Practical 12: Collections Framework**	**16**
14	**Practical 13: Generics and Repository**	**17**
15	**How to Compile and Run**	**18**
16	**Final Touch of Program — Screenshots**	**19**
17	**TXT File Output**	**20**
18	**Conclusion**	**21**

Introduction

MiniBank is a Java-based banking project used to practise core Java and object-oriented programming. It models bank accounts and demonstrates validation, collections, file persistence, generics, and concurrency.

The code samples below are short examples that explain each practical. Match class and method names with your actual project source before final submission if they differ.

Practical 1: Java Basics

Aim: To understand and apply Java basics in the MiniBank project.

Definition / Concept

MiniBank starts execution from the main() method. Java source code is compiled into bytecode and then run by the JVM.

Short Code Example

```
public class MiniBank {
public static void main(String[] args) {
System.out.println("Welcome to MiniBank");
}
}
```

Result / Observation

The program displays a welcome message when it starts.

```
================================
Welcome to MiniBank
Branch: Main Branch
================================
Existing accounts loaded successfully.
```

Practical 2: Classes and Objects

Aim: To understand and apply classes and objects in the MiniBank project.

Definition / Concept

A class is a blueprint; an object is an instance of a class. MiniBank uses account classes to represent different account types.

Short Code Example

```
SavingsAccount account = new SavingsAccount(...);
CurrentAccount current = new CurrentAccount(...);
FixedDepositAccount deposit = new FixedDepositAccount(...);
```

Result / Observation

Different account objects represent different types of bank accounts.

```
--- Account Details ---
1. Owner: Riya | Account Number: AC0001 | Balance: Rs. 4000
2. Owner: Aman | Account Number: AC0002 | Balance: Rs. 8000
3. Owner: Neha | Account Number: AC0003 | Balance: Rs. 10000
```

Practical 3: Constructors and Encapsulation

Aim: To understand and apply constructors and encapsulation in the MiniBank project.

Definition / Concept

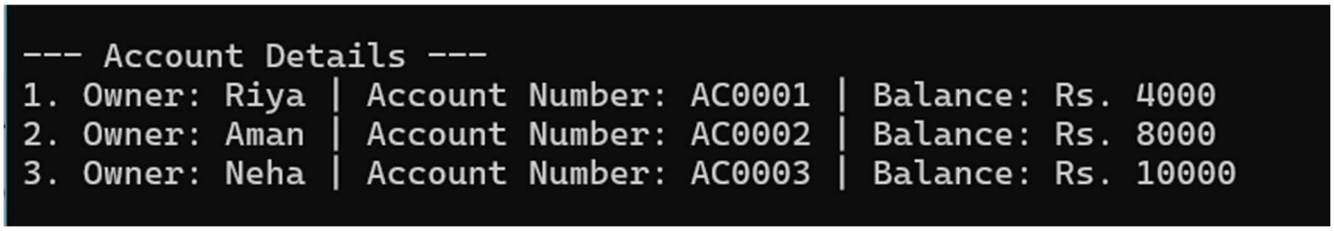
A constructor initializes an object. Encapsulation keeps data controlled inside a class and provides access through methods.

Short Code Example

```
public double getBalance() {
return balance;
}
public String getOwnerName() {
return ownerName;
}
```

Result / Observation

Account details are accessed through methods such as getBalance() and getOwnerName().



Practical 4: Inheritance

Aim: To understand and apply inheritance in the MiniBank project.

Definition / Concept

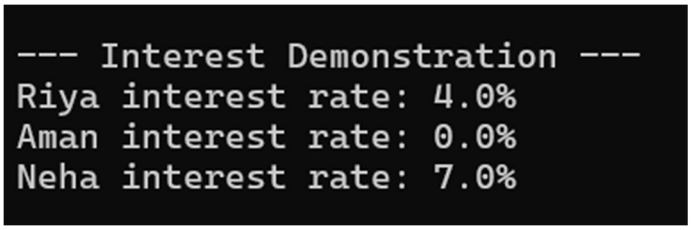
Inheritance lets specialised classes reuse common fields and methods from a parent class.

Short Code Example

```
class SavingsAccount extends Account {
// Savings-account-specific behaviour
}

class CurrentAccount extends Account {
// Current-account-specific behaviour
}
```

Result / Observation



Practical 5: Abstraction and Polymorphism

Aim: To understand and apply abstraction and polymorphism in the MiniBank project.

Definition / Concept

Polymorphism allows a parent-class reference to refer to different child objects. An interface such as InterestBearing describes interest behaviour.

Short Code Example

```
Account account = new SavingsAccount(...);
// The actual account type determines overridden behaviour.

if (account instanceof InterestBearing) {
InterestBearing item = (InterestBearing) account;
// Access interest-related behaviour
}
```

Result / Observation

```
--- Interest Demonstration ---
Riya interest rate: 4.0%
Aman interest rate: 0.0%
Neha interest rate: 7.0%
```

Practical 6: Interfaces and Lambda Expressions

Aim: To understand and apply interfaces and lambda expressions in the MiniBank project.

Definition / Concept

A functional interface has one abstract method and can be implemented using an anonymous class or a lambda expression.

Short Code Example

```
WithdrawRule rule = amount -> amount > 0;
boolean allowed = rule.isAllowed(500);
```

Result / Observation

WithdrawRule is used to check a withdrawal amount. The project demonstrates an anonymous class and a lambda.

```
--- WithdrawRule: Anonymous Class ---
Riya can withdraw Rs. 3000: true

--- WithdrawRule: Lambda ---
Riya can withdraw Rs. 1500: true
Riya can withdraw Rs. 3000: false
```

Practical 7: Annotations and Reflection

Aim: To understand and apply annotations and reflection in the MiniBank project.

Definition / Concept

An annotation adds metadata to code. Reflection allows a program to inspect class fields and annotations at runtime.

Short Code Example

```
@Id
private String accountNumber;

// Reflection can inspect fields:
Field[] fields = Account.class.getDeclaredFields();
```

Result / Observation

The Repository can inspect the @Id field to identify an entity.

Practical 8: Exception Handling

Aim: To understand and apply exception handling in the MiniBank project.

Definition / Concept

Exceptions represent errors that happen while a program runs. try-catch allows the program to handle an error without stopping unexpectedly.

Short Code Example

```
try {
account.withdraw(amount);
} catch (Exception e) {
System.out.println("Transaction failed: " + e.getMessage());
}
```

Result / Observation



Banking errors such as insufficient funds and invalid menu input are handled with useful messages.

Practical 9: Multithreading

Aim: To understand and apply multithreading in the MiniBank project.

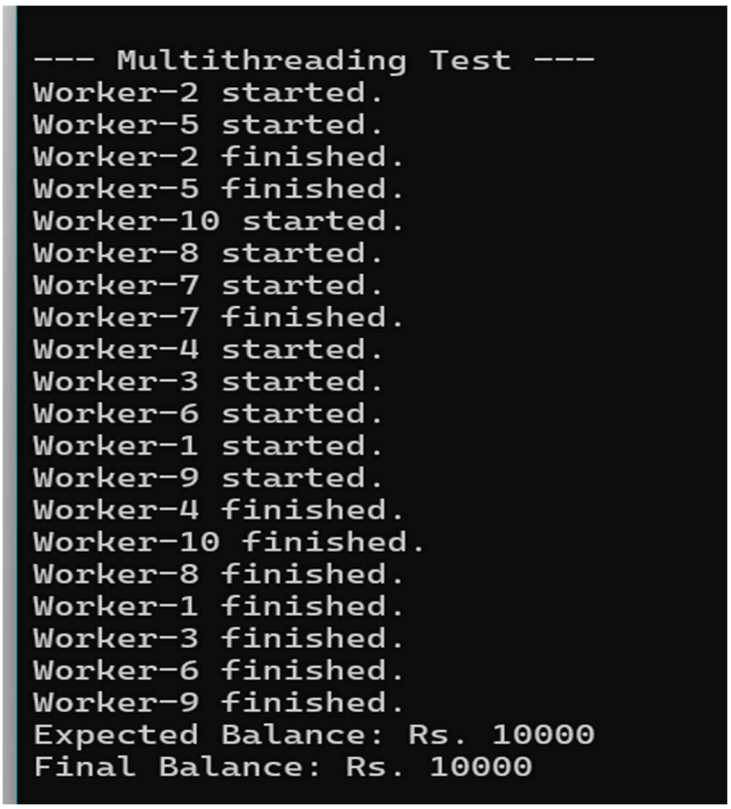
Definition / Concept

Multithreading allows multiple tasks to run concurrently. Shared balance updates should be synchronized to avoid incorrect results.

Short Code Example

```
Thread worker = new Thread(() -> {
// Deposit Rs. 1000
});
worker.start();
```

Result / Observation



Ten worker threads each deposit Rs. 1000. Expected and observed final balance: Rs. 10000.

Practical 10: Thread Pools, Producer–Consumer, and Deadlock

Aim: To understand and apply thread pools, producer–consumer, and deadlock in the MiniBank project.

Definition / Concept

A thread pool reuses worker threads. Producer–consumer passes values through a shared queue. Deadlock can be prevented by acquiring locks in a consistent order.

Short Code Example

```
ExecutorService pool = Executors.newFixedThreadPool(3);
pool.submit(() -> {
// Perform a deposit or withdrawal
});
pool.shutdown();
```

Result / Observation

```
--- Thread Pool Test ---
ConcurrentHashMap lookup: AC0001
pool-1-thread-2 deposited Rs. 100
pool-1-thread-1 deposited Rs. 100
pool-1-thread-3 deposited Rs. 100
pool-1-thread-2 deposited Rs. 100
pool-1-thread-4 deposited Rs. 100
pool-1-thread-1 withdrew Rs. 50
pool-1-thread-3 withdrew Rs. 50
pool-1-thread-2 withdrew Rs. 50
pool-1-thread-4 withdrew Rs. 50
pool-1-thread-1 withdrew Rs. 50

--- Thread Pool Task Counts ---
pool-1-thread-1 executed 3 tasks
pool-1-thread-3 executed 2 tasks
pool-1-thread-2 executed 3 tasks
pool-1-thread-4 executed 2 tasks
Expected Balance: Rs. 1250
Final Balance: Rs. 1250
```

Thread-pool balance: Rs. 1250. Producer–consumer completed ten values, and both deadlock-prevention transfers completed.

```
--- Producer-Consumer Test ---
Produced transaction: 1
Produced transaction: 2
Produced transaction: 3
Produced transaction: 4
Produced transaction: 5
Processed transaction: 1
Processed transaction: 2
Processed transaction: 3
Processed transaction: 4
Processed transaction: 5
Produced transaction: 6
Produced transaction: 7
Produced transaction: 8
Produced transaction: 9
Produced transaction: 10
Processed transaction: 6
Processed transaction: 7
Processed transaction: 8
Processed transaction: 9
Processed transaction: 10
Producer-Consumer test completed.
```

```
--- Deadlock Demonstration ---
Transfer-B-to-A locked AC0008
Transfer-A-to-B locked AC0007
Deadlock detected!
Both threads are waiting for each other's account lock.

--- Deadlock Prevention ---
Fixed-Transfer-A-to-B transferred Rs. 100
Fixed-Transfer-B-to-A transferred Rs. 100
Both transfers completed successfully.
Account C balance: Rs. 1000
Account D balance: Rs. 1000
Transfer test time: 6 ms
```

Practical 11: File Persistence and Reports

Aim: To understand and apply file persistence and reports in the MiniBank project.

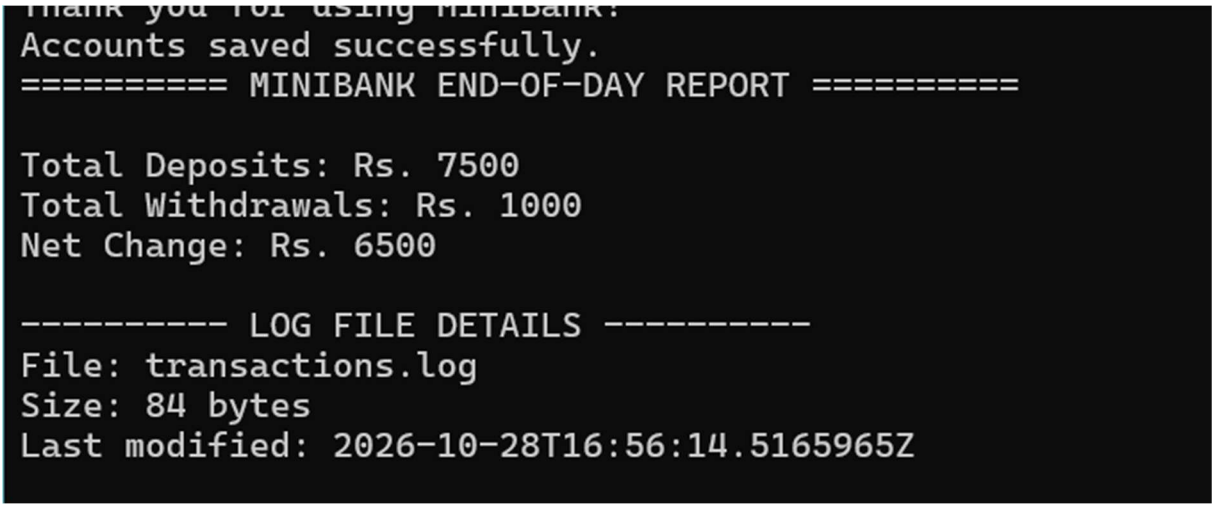
Definition / Concept

Persistence saves data so it can be loaded again later. Serialization can write account objects to a file; transaction logs and end-of-day reports can also be saved.

Short Code Example

```
try (ObjectOutputStream out =
new ObjectOutputStream(new FileOutputStream("accounts.dat"))) {
out.writeObject(account);
}
```

Result / Observation



Accounts are saved and loaded using serialization. Transaction logs and an end-of-day report are stored in files.

Practical 12: Collections Framework

Aim: To understand and apply the Collections Framework in the MiniBank project.

Definition / Concept

Collections store and organise groups of objects. ArrayList stores a list, Comparable defines natural ordering, Comparator defines custom ordering, and ConcurrentHashMap supports concurrent access.

Short Code Example

```
List<Account> accounts = new ArrayList<>();
accounts.sort(Comparator.comparingDouble(Account::getBalance).reversed());
```

```
--- Practical 12: AccountStore ---

Account lookup: AC0002
Account found: AC0002 | Aman | 8000

--- Accounts Sorted by Balance ---
AC0003 | Neha | 10000
AC0002 | Aman | 8000
AC0001 | Riya | 4000

--- Accounts With Balance Above Rs. 2000 ---
AC0001 | Riya | 4000
AC0003 | Neha | 10000
AC0002 | Aman | 8000

--- Natural Ordering: Comparable ---
AC0001 | Riya | 4000
AC0002 | Aman | 8000
AC0003 | Neha | 10000
```

Result / Observation

Accounts were found, filtered, and sorted by balance. AC0001 was found through ConcurrentHashMap.

Practical 13: Generics and Repository

Aim: To understand and apply generics and Repository in the MiniBank project.

Definition / Concept

Generics let a class or method work with a specified type while preserving type safety. Repository<Account> provides common storage operations for accounts.

Short Code Example

```
Repository<Account> repository = new Repository<>();
repository.saveAll(accounts);
Account found = repository.findById("AC0002");
int total = repository.count();
```

Result / Observation

```
--- Practical 13: Generic Repository ---

Account lookup using Repository: AC0002
Account found: AC0002 | Aman | 8000

--- All Repository Accounts ---
AC0001 | Riya | 4000
AC0003 | Neha | 10000
AC0002 | Aman | 8000

Total accounts in Repository: 3

--- Repository Delete Test ---
Test account deleted: true
Remaining test accounts: 0
```

AC0002 was found, repository count was 3, delete test returned true, and averages were 25.0 and 20.5.

How to Compile and Run

Open Command Prompt in the project folder C:\MiniBank.

Compile

```
javac -d out src/model/annotation/*.java src/util/*.java src/model/*.java src/exception/*.java src/service/*.java
```

Run

```
java -cp out service.MiniBank
```

Final Touch Of Program

```
========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice: 1

--- Account Details ---
1. Owner: Riya | Account Number: AC0001 | Balance: Rs. 4500
2. Owner: Aman | Account Number: AC0002 | Balance: Rs. 8000
3. Owner: Neha | Account Number: AC0003 | Balance: Rs. 10000

========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice:
```

```
========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice: 1

--- Account Details ---
1. Owner: Riya | Account Number: AC0001 | Balance: Rs. 4500
2. Owner: Aman | Account Number: AC0002 | Balance: Rs. 8000
3. Owner: Neha | Account Number: AC0003 | Balance: Rs. 10000
```

```
========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice: 2

--- Account Details ---
1. Owner: Riya | Account Number: AC0001 | Balance: Rs. 4500
2. Owner: Aman | Account Number: AC0002 | Balance: Rs. 8000
3. Owner: Neha | Account Number: AC0003 | Balance: Rs. 10000
Select account number (1-3): 3
Enter deposit amount: 5000
Deposit successful.
Current balance: Rs. 15000
```

```
========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice: 3

--- Account Details ---
1. Owner: Riya | Account Number: AC0001 | Balance: Rs. 4500
2. Owner: Aman | Account Number: AC0002 | Balance: Rs. 8000
3. Owner: Neha | Account Number: AC0003 | Balance: Rs. 15000
Select account number (1-3): 2
Enter withdrawal amount: 2000
Withdrawal successful.
Current balance: Rs. 6000
```

```
========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice: 4

--- Account Details ---
1. Owner: Riya | Account Number: AC0001 | Balance: Rs. 4500
2. Owner: Aman | Account Number: AC0002 | Balance: Rs. 6000
3. Owner: Neha | Account Number: AC0003 | Balance: Rs. 15000
Select account number (1-3): 3
Interest rate: 7.0%
```

```
========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice: 5

--- Transaction History ---
DEPOSIT AC0003 5000
WITHDRAW AC0002 2000
```

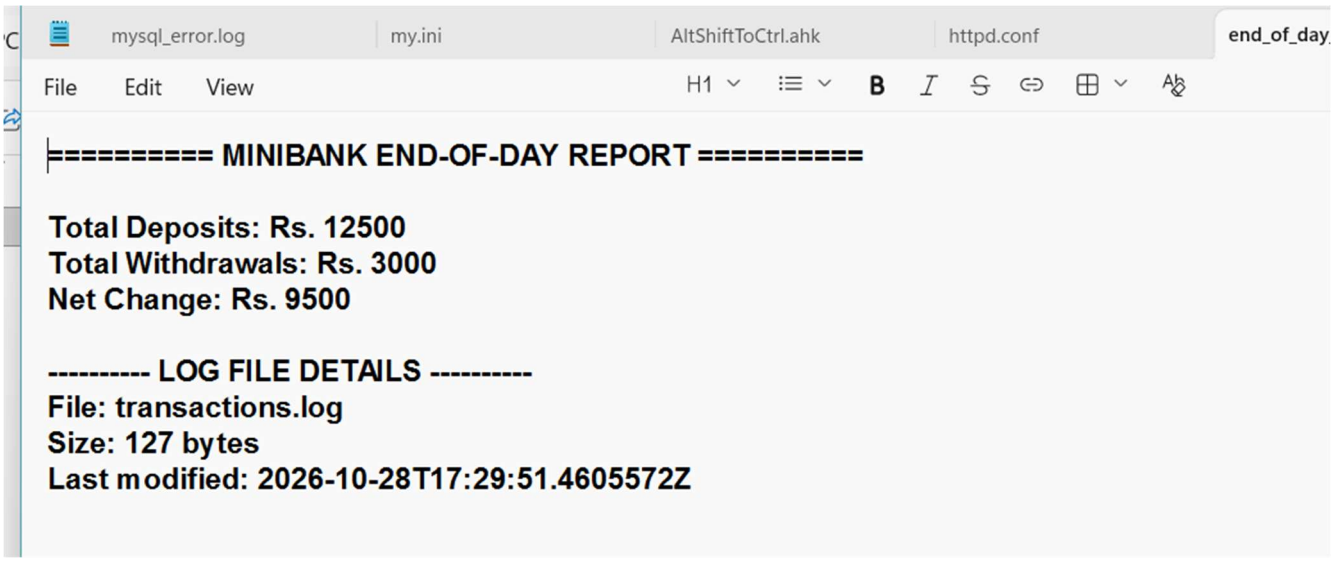
```
========== MINIBANK MENU ==========
1. Display Accounts
2. Deposit
3. Withdraw
4. Check Interest
5. Show Transaction History
6. Exit
Enter your choice: 6
Thank you for using MiniBank!
Accounts saved successfully.
========== MINIBANK END-OF-DAY REPORT ==========

Total Deposits: Rs. 12500
Total Withdrawals: Rs. 3000
Net Change: Rs. 9500

---------- LOG FILE DETAILS ----------
File: transactions.log
Size: 127 bytes
Last modified: 2026-10-28T17:29:51.4605572Z

Report saved to: data\reports\end_of_day_report.txt
```

TXT File O/P



Conclusion

The MiniBank project demonstrates core Java and object-oriented programming concepts through a banking application. The project includes account classes, exception handling, file storage, collections, generics, and concurrent programming. The observed tests showed successful account lookup, expected balances, completion of producer-consumer processing, deadlock prevention, and correct generic repository operations.